Accueil du niveau

FONCTIONS ET STRUCTURES DE CONTRÔLE

Le cerveau de vos programmes

LOGIQUE DU PROGRAMME

1 / 10

Accueil du niveau

CONDITIONS : IF, ELIF, ELSE

Prendre des décisions dans le code

```
age = 17

if age >= 18:
    print("Majeur")
elif age >= 16:
    print("Presque majeur")
else:
    print("Mineur")
# "Presque majeur"

# Opérateur ternaire
statut = "majeur" if age >= 18 else "mineur"

# Conditions combinées
if age >= 16 and age < 18:
    print("Bientôt !")
```

AIGUILLAGE CONDITIONNEL

données ▸ Aiguillage conditionnel ▸ résultat

2 / 10

Accueil du niveau

LA BOUCLE FOR

Répéter pour chaque élément

```
fruits = ["pomme", "banane", "cerise"]

for fruit in fruits:
    print(f"J'aime la {fruit}")

# Parcourir un dictionnaire
scores = {"Alice": 14, "Bob": 17}
for nom, note in scores.items():
    print(f"{nom} : {note}/20")

# Parcourir une chaîne
for lettre in "Python":
    print(lettre, end=" ")
# P y t h o n
```

ITÉRATION FOR

Itération for

Accueil du niveau

LA BOUCLE WHILE

Répéter tant qu'une condition est vraie

```
# Compteur simple
compteur = 0
while compteur < 5:
    print(compteur, end=" ")
    compteur += 1
# 0 1 2 3 4

# break : sortie anticipée
while True:
    reponse = input("Quit? (o/n) ")
    if reponse == "o":
        break

# continue : sauter une itération
for i in range(10):
    if i % 2 == 0:
        continue
    print(i, end=" ")  # 1 3 5 7 9
```

BOUCLE WHILE

Boucle while

Accueil du niveau

4 / 10

Accueil du niveau

RANGE() ET ENUMERATE()

Deux outils indispensables des boucles

```
# range(stop), range(start, stop),
range(start, stop, step)
list(range(5))        # [0, 1, 2, 3, 4]
list(range(2, 8))     # [2, 3, 4, 5, 6, 7]
list(range(0, 10, 3)) # [0, 3, 6, 9]

# enumerate : indice + valeur
fruits = ["pomme", "banane", "cerise"]
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
# 0: pomme
# 1: banane
# 2: cerise

for i, fruit in enumerate(fruits, start=1):
    print(f"{i}. {fruit}")
```

RANGE(0, 5)

```
[0]
```

```
[1]
```

Accueil du niveau

[2]

[3]

5 / 10

Accueil du niveau

DÉFINIR UNE FONCTION

def, le mot magique de la réutilisabilité

```
def saluer(prenom):
    """Affiche un message de bienvenue."""
    print(f"Bonjour {prenom} !")

saluer("Alice")   # Bonjour Alice !
saluer("Bob")     # Bonjour Bob !

def calculer_aire(longueur, largeur):
    """Calcule l'aire d'un rectangle."""
    return longueur * largeur

aire = calculer_aire(5, 3)
print(f"Aire : {aire} m²")  # Aire : 15 m²
```

DEF F(X) → Y

```
def f(x)
```

→

```
y
```

Accueil du niveau

ARGUMENTS PAR DÉFAUT

Flexibilité et valeurs prédéfinies

```
def presenter(nom, ville="Paris", age=None):
    msg = f"{nom} de {ville}"
    if age is not None:
        msg += f", {age} ans"
    return msg

presenter("Alice")              # "Alice de Paris"
presenter("Bob", "Lyon")        # "Bob de Lyon"
presenter("Léa", age=25)        # "Léa de Paris, 25 ans"

# *args : arguments positionnels variables
def somme(*nombres):
    return sum(nombres)

somme(1, 2, 3, 4)  # 10
```

PARAMÈTRES NOMMÉS

"Paramètres nommés_1" →valeur
"Paramètres nommés_2" →valeur

```
"Paramètres nommés_3" →valeur
```

Accueil du niveau

7 / 10

Accueil du niveau

RETURN ET VALEURS DE RETOUR

La sortie de votre machine

```
def analyser_notes(notes):
    """Renvoie min, max et moyenne."""
    return min(notes), max(notes),
sum(notes)/len(notes)

# Déballage des 3 valeurs
note_min, note_max, moyenne =
analyser_notes([14, 8, 17, 12])
print(f"Min={note_min}, Max={note_max}, Moy=
{moyenne:.1f}")
# Min=8, Max=17, Moy=12.8

# Return anticipé (guard clause)
def diviser(a, b):
    if b == 0:
        return None
    return a / b
```

ENTRÉE → SORTIE

~~Entrée~~

→

Accueil du niveau

Sortie

```
8 / 10
```

Accueil du niveau

LES COMPRÉHENSIONS DE LISTE

La formule magique en une ligne

```
# Boucle classique
carres = []
for x in range(10):
    carres.append(x ** 2)

# Compréhension de liste (1 ligne !)
carres = [x ** 2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Avec filtre
pairs = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Dict comprehension
noms = ["Alice", "Bob", "Léa"]
longueurs = {n: len(n) for n in noms}
# {"Alice": 5, "Bob": 3, "Léa": 3}
```

[X FOR X IN ...]

données ▸ [x for x in ...] ▸ résultat

9 / 10

Accueil du niveau

Accueil du niveau

RÉCAPITULATIF

Votre boîte à outils logique

- ✓ if / elif / else : décisions
- ✓ for : parcourir chaque élément
- ✓ while : répéter sous condition
- ✓ range() / enumerate() : compteurs et indices
- ✓ def : créer des fonctions réutilisables
- ✓ return : renvoyer un ou plusieurs résultats
- ✓ Compréhensions : formule magique en une ligne

10 / 10

Speaker notes